

Cartographie à grande échelle

Utiliser QGIS pour générer des centaines de cartes

Dépôt Github :

github.com/paulbradshaw/QGIS_param

En 1 heure vous saurez :

- Comment utiliser la fonctionnalité **Python de QGIS**
- Comment utiliser la **paramétrisation** pour générer une carte pour chaque zone/partenaire/ville
- Conseils pour utiliser l'**IA** pour faciliter le codage

Scénario :

Le BBC Shared Data Unit travaille sur un reportage concernant les défenses contre les inondations. Il sera diffusé dans les rédactions à travers le Royaume-Uni. Nous voulons générer une image de carte pour la rédaction de chaque ville, montrant l'état des défenses contre les inondations dans sa zone de couverture.

**Comment créer
une carte ?**

Faites-le maintenant :
Trouver le fichier
shapefile
Importer dans QGIS

Suivez le guide dans le dépôt

1. Accédez à [AIMS Spatial Flood Defences \(inc. standardised attributes\)](#) et téléchargez le fichier nommé `AIMS_Spatial_Flood_Defences_inc_standardised_attributes.shp.zip`
2. Décompressez ce fichier
3. Ouvrez QGIS et créez un nouveau projet (*Project > New*). Enregistrez-le dans le même dossier.
4. Sélectionnez *Layer > Add Layer > Add vector layer*.
5. Dans la fenêtre qui s'ouvre, cliquez sur le ... à côté de la section Source (où il est indiqué *Vector Dataset(s)*) et localisez le fichier dans le dossier téléchargé se terminant par **.shp**. Cliquez sur **Ajouter**. La couche devrait apparaître en arrière-plan. Cliquez sur **OK** puis sur **Fermer** pour y revenir.
6. Dans la zone *Navigateur* en haut à gauche de l'écran, faites défiler vers le bas jusqu'à la section Tuiles XYZ. Double-cliquez sur OpenStreetMap pour ajouter ce fond de carte à votre projet (cliquez sur OK pour approuver le changement de SCR). Il devrait maintenant apparaître dans votre panneau de Couches.
7. Cliquez et faites glisser la couche OSM pour la placer sous la couche des défenses contre les inondations.

Personnaliser l'apparence

1. Faites un clic droit sur la couche des défenses contre les inondations et sélectionnez **Propriétés...**
2. Sélectionnez **Symbologie** pour modifier la couleur, l'opacité, l'épaisseur, le style, etc. des lignes/formes

**Comment exporter
une image ?**

Faites-le maintenant :
Zoomer sur une zone
Project > New Print
Layout...

Créer une mise en page (nous allons automatiser cela)

1. Zoomez sur la partie de la carte dont vous souhaitez créer une image (par exemple, zoomée sur une ville)
2. Allez dans **Project > New Print Layout...** Donnez-lui un nom (par ex. cityzoom) et cliquez sur OK
3. La fenêtre de mise en page apparaîtra vide. Cliquez sur le bouton Ajouter une carte sur la gauche (icône de page avec un signe plus vert)
4. Cliquez et faites glisser pour dessiner la zone où vous souhaitez que votre carte apparaisse.
5. Allez dans l'onglet **Propriétés de l'élément** sur le côté droit et faites défiler jusqu'à **Position et taille**. Modifiez-le selon les dimensions souhaitées pour votre image (changez l'unité en pixels si c'est pour le web).
6. Passez à l'onglet **Mise en page** juste à côté, et faites défiler vers le bouton **Redimensionner la mise en page**. Cliquez dessus. L'image devrait maintenant occuper tout le canevas (vous pouvez zoomer avec les boutons de zoom dans le coin supérieur gauche)
7. Lorsque vous êtes satisfait de l'image, allez dans **Layout > Export as Image**
8. Fermez la fenêtre de mise en page et revenez au projet QGIS



**Comment faire
cela **plusieurs fois**
?**

Si vous devez faire quelque chose plus d'une fois...

Plugins > Python Console

Écrivez/ouvrez Python dans la Console **sous la carte.**

*Cliquez sur le bouton **Afficher l'éditeur** pour ouvrir/modifier des fichiers .py*

PyQGIS library ([documentation](#))

Faites-le maintenant :
Téléchargez le fichier
.py

Ouvrez-le dans votre
Console

Standardise

Python Console

```
1 # Python Console
2 # Use iface to access QGIS API interface or type '?' for more info
3 # Security warning: typing commands from an untrusted source can harm your computer
4
```

>>>



Untitled-0 X

1

Coordinate

357074,195749



Scale

1:152681



Magnifier

100%

Rotation

0.0 °



Render



EPSG:27700



(Étapes de la partie 3 du guide dans le dépôt)

1. Accédez au [fichier brut](#) sur GitHub. Cliquez sur Fichier > Enregistrer sous... dans votre navigateur et enregistrez le fichier dans votre dossier de projet
2. Décommentez cette ligne et modifiez le chemin pour pointer vers votre dossier de projet

```
# out_prefix = "/Users/paul/Downloads/testqgis/images/"
```

(Pour trouver le chemin de votre dossier sur Mac, faites un clic droit dessus et sélectionnez Lire les informations. La fenêtre qui apparaît vous indiquera le chemin sous 'Emplacement :')

3. Dans la console Python de QGIS, ouvrez le fichier, puis exécutez le code

```
# Charger les outils pour contrôler les cartes et mises en page QGIS depuis ce script
```

```
from qgis.PyQt.QtCore import QRectF
```

```
from qgis.core import (
```

```
    QgsProject, QgsCoordinateReferenceSystem, QgsPrintLayout, QgsLayoutItemMap,
```

```
    QgsLayoutPoint, QgsLayoutSize, QgsUnitTypes, QgsLayoutItemLabel,
```

```
    QgsLayoutExporter, QgsRectangle, QgsCoordinateTransform, QgsPointXY
```

```
)
```

```
# Charger math pour les chiffres, re (regex) pour le texte, os pour la navigation de fichiers
```

```
import math, re
```

```
import os
```

C'est ici que le dossier de sortie est défini

```
# Utiliser la bibliothèque os pour créer un dossier sur votre ordinateur pour y
enregistrer les images de carte exportées.

# Il sera placé dans votre dossier Téléchargements, dans un sous-dossier appelé
"qgis_images".

# Si le dossier existe déjà, cela ne fait rien – aucune donnée ne sera écrasée.

out_dir = os.path.join(os.path.expanduser("~"), "Downloads", "qgis_images")

os.makedirs(out_dir, exist_ok=True)

out_prefix = out_dir + "/"

# Pour remplacer le dossier configuré ci-dessus, décommentez et adaptez la ligne ci-
dessous.

# out_prefix = "/Users/paul/Downloads/testqgis/images/"
```

#Définir le système de référence de coordonnées (CRS) - EPSG:4326 est utilisé pour le GPS
#mais le 27700 est utilisé pour les défenses contre les inondations au Royaume-Uni, donc nous
l'utilisons pour plus de rigueur

```
QgsProject.instance().setCrs(QgsCoordinateReferenceSystem("EPSG:27700"))
```

Récupérer le projet QGIS actuellement ouvert et le stocker dans une variable appelée project

```
project = QgsProject.instance()
```

#Utiliser EPSG:27700 signifie que nous devons convertir les lat/long en coordonnées BNG

#définir les systèmes source (EPSG:4326) et de destination (EPSG:27700)

```
source_crs = QgsCoordinateReferenceSystem("EPSG:4326")
```

```
dest_crs = QgsCoordinateReferenceSystem("EPSG:27700")
```

#puis créer le transformateur qui utilise ces deux systèmes

```
transformer = QgsCoordinateTransform(  
    source_crs,  
    dest_crs,  
    project  
)
```

#créer une nouvelle mise en page d'impression, dans ce projet, la stocker dans une variable appelée layout

layout = QgsPrintLayout(project)

#remplir avec les paramètres par défaut

layout.initializeDefaults()

#lui donner un nom dans QGIS

layout.setName("centres_layout") # Give the layout a name

Ajouter un panneau de carte à la mise en page – c'est le rectangle qui affichera réellement la carte.

```
map_item = QgsLayoutItemMap(layout)
```

```
map_item.setFrameEnabled(True) # Dessiner une bordure autour du panneau de carte
```

Positionner le panneau de carte à 20mm du bord gauche et 20mm du bord supérieur de la page.

```
map_item.attemptMove(QgsLayoutPoint(20, 20, QgsUnitTypes.LayoutMillimeters))
```

Dimensionner le panneau de carte : 257mm de large et 170mm de haut.

```
map_item.attemptResize(QgsLayoutSize(257, 170, QgsUnitTypes.LayoutMillimeters))
```

```
layout.addLayoutItem(map_item) # Add the map panel to the layout page
```

Ajouter une étiquette texte à la mise en page, qui affichera le nom et les coordonnées de chaque emplacement.

```
label = QgsLayoutItemLabel(layout)
```

```
layout.addLayoutItem(label)
```

Positionner l'étiquette près du haut de la page (20mm du bord gauche, 5mm du bord supérieur).

```
label.attemptMove(QgsLayoutPoint(20, 5, QgsUnitTypes.LayoutMillimeters))
```

def crée une fonction (une recette) que nous pourrons utiliser ultérieurement. Cette fonction calcule une zone rectangulaire sur une carte centrée sur un point donné. Elle nécessite quatre paramètres : les coordonnées x et y du point central ; et la moitié de la largeur/hauteur (une valeur par défaut est utilisée pour le dernier paramètre si celui-ci n'est pas fourni).

```
def rect_around(x, y, half_width, half_height=None):
```

```
    # Si aucune hauteur n'est fournie, utiliser la même valeur que la largeur
```

```
    if half_height is None:
```

```
        half_height = half_width
```

```
    # Lors de l'utilisation, cela retourne un rectangle pour définir l'étendue de la carte.
```

```
    return QgsRectangle(
```

```
        x - half_width,
```

```
        y - half_height,
```

```
        x + half_width,
```

```
        y + half_height )
```

C'est ici que les **emplacements** sont enregistrés

Créer une liste d'emplacements à cartographier. Chaque élément a une latitude, une longitude et un nom.

On commence avec seulement deux éléments pour les tests, puis on l'étendra à tous les emplacements souhaités

```
listofdicstocsv = [  
    {'lat': 50.844441271809465, 'long': -0.2985307978506628, 'officialname': 'Adur  
District Council'},  
    {'lat': 54.71108952940033, 'long': -3.2472347297148736, 'officialname':  
'Allerdale Borough Council'}  
]
```

C'est ici que l'échelle est définie

DÉFINIR L'ÉCHELLE DE LA CARTE : 1 unité sur la carte équivaut à 175 000 dans la réalité

MAP_SCALE = 175000

C'est ici que la **sortie** est définie

```
# Configurer l'outil qui enregistrera la mise en page sous forme d'image.
```

```
exporter = QgsLayoutExporter(layout)
```

```
settings = QgsLayoutExporter.ImageExportSettings()
```

```
# DÉFINIR LA TAILLE DE L'IMAGE DE SORTIE
```

```
# 1200x795px pour utilisation web, correspondant au ratio A4 du code  
précédent
```

```
settings.imageSize = QSize(1200, 795)
```

```
# Parcourir chaque emplacement dans la liste et utiliser la fonction pour créer/exporter une image
for rec in listofdicstocsv:

    # Lire la latitude, la longitude et le nom du dictionnaire actuel

    lat = rec.get('lat', float('nan')) # 'nan' utilisé comme substitut si la valeur est manquante
    lon = rec.get('long', float('nan'))

    officialname = rec.get('officialname', 'unknown')

    # Si la latitude ou la longitude est manquante, ignorer cet emplacement
    if math.isnan(lat) or math.isnan(lon):

        print(f"skipping '{officialname}': missing lat/long")

        continue

    # Déplacer la vue de la carte pour la centrer sur l'emplacement actuel
    transformed_point = transformer.transform( QgsPointXY(lon, lat) )

    # suite...
```

```
# Extraire les coordonnées transformées

x = transformed_point.x()
y = transformed_point.y()

#Créer un rectangle temporaire autour du point pour le centrer
map_item.setExtent( rect_around(x, y, 1000, 1000) )

#Définir une échelle pour contrôler le zoom. Chaque carte utilisera cela.
map_item.setScale(MAP_SCALE)

#ajouter une étiquette avec ces informations
label.setText( f"{officialname} | scale 1:{MAP_SCALE:,.}" )

#actualiser la carte avant l'export
map_item.refresh()

# suite...
```

```
#donner un nom de fichier - supprimer les caractères problématiques
safe_name = re.sub( r'^A-Za-z0-9._-]+', '_', officialname).strip('_')
#utiliser cela pour déterminer le chemin de sauvegarde
out_path = ( f"{out_prefix}" f"{safe_name}.png" )
#et exporter vers ce chemin
result = exporter.exportToImage( out_path, settings )
#si ce n'est pas un succès
if result != QgsLayoutExporter.Success:
    #afficher un message d'erreur
    raise RuntimeError(f"Export failed for {out_path}")
#afficher un message à la fin de chaque boucle
print("wrote", out_path)
```

Comment faire
ça 258 fois ?

Récap : où les emplacements sont enregistrés

Créer une liste d'emplacements à cartographier. Chaque élément a une latitude, une longitude et un nom.

On commence avec seulement deux éléments pour les tests, puis on l'étendra à tous les emplacements souhaités

```
listofdicstocsv = [  
    {'lat': 50.844441271809465, 'long': -0.2985307978506628, 'officialname': 'Adur  
District Council'},  
    {'lat': 54.71108952940033, 'long': -3.2472347297148736, 'officialname':  
'Allerdale Borough Council'}  
]
```

Convertir un CSV en liste de dictionnaires

1. **Téléchargez le CSV des villes** (cliquez sur les trois points en haut à droite)
2. Tapez cette formule dans la cellule **J2** :
`=("{ 'lat': "&B2&", 'long': "&C2&", 'officialname': '"&A2&"' }"`
3. Copiez la formule pour chaque ligne. Vous avez maintenant une colonne de dictionnaires.
4. Utilisez cette formule pour assembler les résultats en séparant chaque élément par une virgule
`=TEXTJOIN(" , " , TRUE , J:J)`
5. Dans votre code Python, trouvez les crochets ouvrant et fermant **[crochets]** contenant votre liste de deux éléments, et remplacez ces deux éléments par la nouvelle liste de dictionnaires, en conservant les crochets.

{lat: 51.5072, 'long': -0.1275, 'officialname': 'London'}, {lat: 52.48, 'long': -1.9025, 'officialname': 'Birmingham'}, {lat: 50.8058, 'long': -1.0872, 'officialname': 'Portsmouth'}, {lat: 50.9025, 'long': -1.4042, 'officialname': 'Southampton'}, {lat: 52.9561, 'long': -1.1512, 'officialname': 'Nottingham'}, {lat: 51.4536, 'long': -2.5975, 'officialname': 'Bristol'}, {lat: 53.479, 'long': -2.2452, 'officialname': 'Manchester'}, {lat: 53.4094, 'long': -2.9785, 'officialname': 'Liverpool'}, {lat: 52.6344, 'long': -1.1319, 'officialname': 'Leicester'}, {lat: 50.8147, 'long': -0.3714, 'officialname': 'Worthing'}, {lat: 52.4081, 'long': -1.5106, 'officialname': 'Coventry'}, {lat: 54.5964, 'long': -5.93, 'officialname': 'Belfast'}, {lat: 53.8, 'long': -1.75, 'officialname': 'Bradford'}, {lat: 52.9247, 'long': -1.478, 'officialname': 'Derby'}, {lat: 50.3714, 'long': -4.1422, 'officialname': 'Plymouth'}, {lat: 51.4947, 'long': -0.1353, 'officialname': 'Westminster'}, {lat: 52.5833, 'long': -2.1333, 'officialname': 'Wolverhampton'}, {lat: 52.2304, 'long': -0.8938, 'officialname': 'Northampton'}, {lat: 52.6286, 'long': 1.2928, 'officialname': 'Norwich'}, {lat: 51.8783, 'long': -0.4147, 'officialname': 'Luton'}, {lat: 52.413, 'long': -1.778, 'officialname': 'Solihull'}, {lat: 51.544, 'long': -0.1027, 'officialname': 'Islington'}, {lat: 57.15, 'long': -2.11, 'officialname': 'Aberdeen'}, {lat: 51.3727, 'long': -0.1099, 'officialname': 'Croydon'}, {lat: 50.72, 'long': -1.88, 'officialname': 'Bournemouth'}, {lat: 51.58, 'long': 0.49, 'officialname': 'Basildon'}, {lat: 51.272, 'long': -0.529, 'officialname': 'Maidstone'}, {lat: 51.5575, 'long': 0.0858, 'officialname': 'Ilford'}, {lat: 53.39, 'long': -2.59, 'officialname': 'Warrington'}, {lat: 51.75, 'long': -1.25, 'officialname': 'Oxford'}, {lat: 51.5836, 'long': -0.3464, 'officialname': 'Harrow'}, {lat: 52.519, 'long': -1.995, 'officialname': 'West Bromwich'}, {lat: 51.8667, 'long': -2.25, 'officialname': 'Gloucester'}, {lat: 53.96, 'long': -1.08, 'officialname': 'York'}, {lat: 53.8142, 'long': -3.0503, 'officialname': 'Blackpool'}, {lat: 53.4083, 'long': -2.1494, 'officialname': 'Stockport'}, {lat: 53.424, 'long': -2.322, 'officialname': 'Sale'}, {lat: 51.5975, 'long': -0.0681, 'officialname': 'Tottenham'}, {lat: 52.2053, 'long': 0.1192, 'officialname': 'Cambridge'}, {lat: 51.5768, 'long': 0.1801, 'officialname': 'Romford'}, {lat: 51.8917, 'long': 0.903, 'officialname': 'Colchester'}, {lat: 51.6287, 'long': -0.7482, 'officialname': 'High Wycombe'}, {lat: 54.9556, 'long': -1.6, 'officialname': 'Gateshead'}, {lat: 51.5084, 'long': 0.1192, 'officialname': 'Slough'}, {lat: 53.748, 'long': -2.482, 'officialname': 'Blackburn'}, {lat: 51.73, 'long': 0.48, 'officialname': 'Chelmsford'}, {lat: 53.61, 'long': -2.16, 'officialname': 'Rochdale'}, {lat: 53.43, 'long': -1.357, 'officialname': 'Rotherham'}, {lat: 51.584, 'long': -0.021, 'officialname': 'Walthamstow'}, {lat: 51.2667, 'long': -1.0876, 'officialname': 'Basingstoke'}, {lat: 53.483, 'long': -2.2931, 'officialname': 'Salford'}, {lat: 51.4668, 'long': -0.375, 'officialname': 'Hounslow'}, {lat: 51.5528, 'long': -0.2979, 'officialname': 'Wembley'}, {lat: 52.1911, 'long': -2.2206, 'officialname': 'Worcester'}, {lat: 51.4928, 'long': -0.2229, 'officialname': 'Hammersmith'}, {lat: 51.5864, 'long': 0.6049, 'officialname': 'Rayleigh'}, {lat: 51.7526, 'long': -0.4692, 'officialname': 'Hemel Hempstead'}, {lat: 51.38, 'long': -2.36, 'officialname': 'Bath'}, {lat: 51.5127, 'long': -0.4211, 'officialname': 'Hayes'}, {lat: 54.527, 'long': -1.5526, 'officialname': 'Darlington'}, {lat: 50.8352, 'long': -0.1758, 'officialname': 'Hove'}, {lat: 50.85, 'long': 0.57, 'officialname': 'Hastings'}, {lat: 51.655, 'long': -0.3957, 'officialname': 'Watford'}, {lat: 51.9017, 'long': -0.2019, 'officialname': 'Stevenage'}, {lat: 54.69, 'long': -1.21, 'officialname': 'Hartlepool'}, {lat: 53.19, 'long': -2.89, 'officialname': 'Chester'}, {lat: 51.4828, 'long': -0.195, 'officialname': 'Fulham'}, {lat: 52.523, 'long': -1.468, 'officialname': 'Nuneaton'}, {lat: 51.5175, 'long': -0.2988, 'officialname': 'Ealing'}, {lat: 51.8168, 'long': -0.8124, 'officialname': 'Aylesbury'}, {lat: 51.6154, 'long': -0.0708, 'officialname': 'Edmonton'}, {lat: 51.755, 'long': -0.336, 'officialname': 'Saint Albans'}, {lat: 53.789, 'long': -2.248, 'officialname': 'Burnley'}, {lat: 53.7167, 'long': -1.6356, 'officialname': 'Batley'}, {lat: 53.5809, 'long': -0.6502, 'officialname': 'Scunthorpe'}, {lat: 52.508, 'long': -2.089, 'officialname': 'Dudley'}, {lat: 51.4575, 'long': -0.1175, 'officialname': 'Brixton'}, {lat: 51.5111, 'long': -0.3756, 'officialname': 'Southall'}, {lat: 55.8456, 'long': -4.4239, 'officialname': 'Paisley'}, {lat: 51.37, 'long': 0.52, 'officialname': 'Chatham'}, {lat: 53.523, 'long': 0.0554, 'officialname': 'East Ham'}, {lat: 51.346, 'long': -2.977, 'officialname': 'Weston-super-Mare'}, {lat: 54.8947, 'long': -2.9364, 'officialname': 'Carlisle'}, {lat: 54.995, 'long': -1.43, 'officialname': 'South Shields'}, {lat: 55.7644, 'long': -4.1769, 'officialname': 'East Kilbride'}, {lat: 52.8019, 'long': -1.6367, 'officialname': 'Burton upon Trent'}, {lat: 53.9919, 'long': -1.5378, 'officialname': 'Harrogate'}, {lat: 53.099, 'long': -2.44, 'officialname': 'Crewe'}, {lat: 52.48, 'long': 1.75, 'officialname': 'Lewes'}, {lat: 52.37, 'long': -1.26, 'officialname': 'Rugby'}, {lat: 51.623, 'long': 0.009, 'officialname': 'Chingford'}, {lat: 51.5404, 'long': -0.4778, 'officialname': 'Uxbridge'}, {lat: 52.58, 'long': -1.98, 'officialname': 'Walsall'}, {lat: 51.475, 'long': 0.33, 'officialname': 'Grays'}, {lat: 51.3868, 'long': -0.4133, 'officialname': 'Walton upon Thames'}, {lat: 51.4002, 'long': -0.1086, 'officialname': 'Thornthorpe Heath'}, {lat: 51.599, 'long': -0.187, 'officialname': 'Finchley'}, {lat: 51.5, 'long': -0.19, 'officialname': 'Kensington'}, {lat: 52.974, 'long': -0.0214, 'officialname': 'Boston'}, {lat: 50.4353, 'long': -3.5625, 'officialname': 'Paignton'}, {lat: 50.88, 'long': -1.03, 'officialname': 'Waterlooville'}, {lat: 53.875, 'long': -1.706, 'officialname': 'Guisely'}, {lat: 51.5565, 'long': 0.2128, 'officialname': 'Hornchurch'}, {lat: 51.4009, 'long': -0.1517, 'officialname': 'Mitcham'}, {lat: 51.4496, 'long': -0.4089, 'officialname': 'Feltham'}, {lat: 52.4575, 'long': -2.1479, 'officialname': 'Stourbridge'}, {lat: 51.375, 'long': 0.5, 'officialname': 'Rochester'}, {lat: 53.691, 'long': -1.633, 'officialname': 'Dewsbury'}, {lat: 51.5135, 'long': -0.2707, 'officialname': 'Acton'}, {lat: 51.449, 'long': -0.337, 'officialname': 'Twickenham'}, {lat: 53.046, 'long': -2.993, 'officialname': 'Wrexham'}, {lat: 53.279, 'long': -2.897, 'officialname': 'Elesmere Port'}, {lat: 54.66, 'long': -5.67, 'officialname': 'Bangor'}, {lat: 51.019, 'long': -3.1, 'officialname': 'Taunton'}, {lat: 52.7225, 'long': -1.2078, 'officialname': 'Loughborough'}, {lat: 51.54, 'long': 0.08, 'officialname': 'Barking'}, {lat: 51.6185, 'long': -0.2729, 'officialname': 'Edgware'}, {lat: 50.8094, 'long': -0.5409, 'officialname': 'Littlehampton'}, {lat: 51.576, 'long': -0.433, 'officialname': 'Ruislip'}, {lat: 51.4279, 'long': -0.1235, 'officialname': 'Streatham'}, {lat: 51.132, 'long': 0.263, 'officialname': 'Royal Tunbridge Wells'}, {lat: 53.35, 'long': -3.003, 'officialname': 'Bebington'}, {lat: 53.25, 'long': -2.13, 'officialname': 'Macclesfield'}, {lat: 52.3028, 'long': -0.6944, 'officialname': 'Wellingborough'}, {lat: 52.3931, 'long': -0.7229, 'officialname': 'Kettering'}, {lat: 51.878, 'long': 0.55, 'officialname': 'Brentree'}, {lat: 52.2919, 'long': -1.5358, 'officialname': 'Royal Leamington Spa'}, {lat: 54.1108, 'long': -3.2261, 'officialname': 'Barrow in Furness'}, {lat: 56.0719, 'long': -3.4393, 'officialname': 'Dunfermline'}, {lat: 53.3838, 'long': -2.3547, 'officialname': 'Altrincham'}, {lat: 54.0489, 'long': -2.8014, 'officialname': 'Lancaster'}, {lat: 53.4872, 'long': -3.0343, 'officialname': 'Crosby'}, {lat: 53.4457, 'long': -2.9891, 'officialname': 'Bootle'}, {lat: 51.5423, 'long': -0.0026, 'officialname': 'Stratford'}, {lat: 51.0792, 'long': 1.1794, 'officialname': 'Fekstone'}, {lat: 55.945, 'long': -3.994, 'officialname': 'Cumbemauld'}, {lat: 51.208, 'long': -1.48, 'officialname': 'Andover'}, {lat: 51.66, 'long': -3.81, 'officialname': 'Neath'}, {lat: 52.488, 'long': -2.05, 'officialname': 'Rowley Regis'}, {lat: 54.2825, 'long': -0.4, 'officialname': 'Scarborough'}, {lat: 55.98, 'long': -3.17, 'officialname': 'Leith'}, {lat: 50.9452, 'long': -2.637, 'officialname': 'Yeovil'}, {lat: 51.451, 'long': 0.052, 'officialname': 'Eltham'}, {lat: 51.5541, 'long': -0.1744, 'officialname': 'Hampstead'}, {lat: 53.125, 'long': -1.261, 'officialname': 'Sutton in Ashfield'}, {lat: 51.4015, 'long': -0.1949, 'officialname': 'Morden'}, {lat: 51.6444, 'long': -0.1997, 'officialname': 'Barnet'}, {lat: 53.4466, 'long': -2.3086, 'officialname': 'Stretford'}, {lat: 51.408, 'long': -0.022, 'officialname': 'Beckenham'}, {lat: 51.5299, 'long': -0.3488, 'officialname': 'Greenford'}, {lat: 51.702, 'long': -0.035, 'officialname': 'Cheshunt'}, {lat: 53.48, 'long': -2.89, 'officialname': 'Kirkby'}, {lat: 51.07, 'long': -1.79, 'officialname': 'Salsbury'}, {lat: 53.4897, 'long': -2.0952, 'officialname': 'Ashton'}, {lat: 51.394, 'long': -0.307, 'officialname': 'Sutton'}, {lat: 53.716, 'long': -1.356, 'officialname': 'Castleford'}, {lat: 51.4452, 'long': -0.0207, 'officialname': 'Catford'}, {lat: 53.3042, 'long': -1.1244, 'officialname': 'Workshop'}, {lat: 53.7492, 'long': -1.6023, 'officialname': 'Morley'}, {lat: 51.743, 'long': -3.378, 'officialname': 'Merthyr Tydfil'}, {lat: 53.555, 'long': -2.187, 'officialname': 'Middleton'}, {lat: 51.2834, 'long': -0.8456, 'officialname': 'Fleet'}, {lat: 50.85, 'long': -1.18, 'officialname': 'Fareham'}, {lat: 53.4487, 'long': -2.3747, 'officialname': 'Urmston'}, {lat: 51.3656, 'long': -0.1963, 'officialname': 'Sutton'}, {lat: 51.578, 'long': -3.218, 'officialname': 'Caerphilly'}, {lat: 51.128, 'long': -2.993, 'officialname': 'Bridgwater'}, {lat: 51.401, 'long': -1.323, 'officialname': 'Newbury'}, {lat: 51.4594, 'long': 0.1097, 'officialname': 'Welling'}, {lat: 51.46, 'long': -2.505, 'officialname': 'Kingswood'}, {lat: 51.886, 'long': -0.521, 'officialname': 'Dunstable'}, {lat: 51.336, 'long': 1.416, 'officialname': 'Ramsgate'}, {lat: 51.393, 'long': 0.478, 'officialname': 'Strood'}, {lat: 53.5533, 'long': -0.0215, 'officialname': 'Cleethorpes'}, {lat: 51.5932, 'long': -0.3894, 'officialname': 'Pinner'}, {lat: 52.606, 'long': 1.729, 'officialname': 'Great Yarmouth'}, {lat: 52.9711, 'long': -1.3092, 'officialname': 'Ilkeston'}, {lat: 53.653, 'long': -2.632, 'officialname': 'Chorley'}, {lat: 51.37, 'long': 1.13, 'officialname': 'Heme Bay'}, {lat: 51.872, 'long': 0.1725, 'officialname': 'Bishops Cleeve'}, {lat: 53.005, 'long': -1.127, 'officialname': 'Arnold'}, {lat: 52.724, 'long': -1.369, 'officialname': 'Coalville'}, {lat: 51.994, 'long': -0.732, 'officialname': 'Bletchley'}, {lat: 51.9165, 'long': -0.6617, 'officialname': 'Leighton Buzzard'}, {lat: 55.86, 'long': -3.98, 'officialname': 'Ardrie'}, {lat: 55.126, 'long': -1.514, 'officialname': 'Blyth'}, {lat: 51.574, 'long': 0.4181, 'officialname': 'Laindon'}, {lat: 51.684, 'long': -4.163, 'officialname': 'Llanelli'}, {lat: 52.927, 'long': -1.215, 'officialname': 'Beeston'}, {lat: 52.4629, 'long': -1.8542, 'officialname': 'South Heath'}, {lat: 55.0456, 'long': -1.4443, 'officialname': 'Whitley Bay'}, {lat: 53.4554, 'long': -2.112, 'officialname': 'Denton'}, {lat: 52.932, 'long': -1.127, 'officialname': 'West Bridgford'}, {lat: 51.6578, 'long': -0.2722, 'officialname': 'Borehamwood'}, {lat: 56.0011, 'long': -3.7835, 'officialname': 'Falkirk'}, {lat: 53.5239, 'long': -2.3991, 'officialname': 'Walden'}, {lat: 51.5878, 'long': -0.3086, 'officialname': 'Kenton'}, {lat: 54.0819, 'long': -0.1923, 'officialname': 'Bridlington'}, {lat: 54.61, 'long': -1.27, 'officialname': 'Billingham'}, {lat: 52.918, 'long': -0.638, 'officialname': 'Grantham'}, {lat: 55.0097, 'long': -1.4448, 'officialname': 'North Shields'}, {lat: 51.947, 'long': -0.283, 'officialname': 'Hitchin'}, {lat: 52.7858, 'long': -0.1529, 'officialname': 'Spalding'}, {lat: 51.36, 'long': 0.61, 'officialname': 'Rainham'}, {lat: 51.978, 'long': -0.23, 'officialname': 'Letchworth'}, {lat: 51.6114, 'long': 0.5207, 'officialname': 'Wickford'}, {lat: 53.4111, 'long': -2.8403, 'officialname': 'Huyton'}, {lat: 51.6717, 'long': -1.2763, 'officialname': 'Abingdon'}, {lat: 51.32, 'long': -2.208, 'officialname': 'Trowbridge'}, {lat: 52.5812, 'long': -1.093, 'officialname': 'Wigston Magna'}, {lat: 51.606, 'long': -1.241, 'officialname': 'Didcot'}, {lat: 51.433, 'long': -0.933, 'officialname': 'Eaerley'}, {lat: 51.459, 'long': 0.138, 'officialname': 'Bexleyheath'}, {lat: 53.4429, 'long': -1.4698, 'officialname': 'Ecclesfield'}, {lat: 53.698, 'long': -2.461, 'officialname': 'Darwen'}, {lat: 53.5333, 'long': -2.2833, 'officialname': 'Prestwich'}, {lat: 51.602, 'long': -3.342, 'officialname': 'Pontypridd'}, {lat: 55.828, 'long': -4.214, 'officialname': 'Rutherglen'}, {lat: 51.1295, 'long': 1.3089, 'officialname': 'Dover'}, {lat: 50.8365, 'long': -0.7792, 'officialname': 'Chichester'}, {lat: 51.2226, 'long': 1.4006, 'officialname': 'Deal'}, {lat: 51.9, 'long': -1.15, 'officialname': 'Bicester'}, {lat: 51.5467, 'long': -0.37, 'officialname': 'Northolt'}, {lat: 55.7742, 'long': -3.9183, 'officialname': 'Wishaw'}, {lat: 51.3652, 'long': -0.1676, 'officialname': 'Carshalton'}, {lat: 53.001, 'long': -1.197, 'officialname': 'Bulwell'}, {lat: 54.591, 'long': -5.68, 'officialname': 'Newtownards'}, {lat: 54.326, 'long': -2.745, 'officialname': 'Kendal'}, {lat: 55.082, 'long': -1.585, 'officialname': 'Cramlington'}, {lat: 52.3353, 'long': -2.0579, 'officialname': 'Bromsgrove'}, {lat: 51.703, 'long': -3.041, 'officialname': 'Pont-y-pŵll'}, {lat: 51.509, 'long': -0.338, 'officialname': 'Hanwell'}, {lat: 51.2279, 'long': -2.3215, 'officialname': 'Frome'}, {lat: 51.5981, 'long': -0.1149, 'officialname': 'Wood Green'}, {lat: 52.5708, 'long': -2.0457, 'officialname': 'Darlaston'}, {lat: 55.181, 'long': -1.568, 'officialname': 'Ashington'}, {lat: 52.9677, 'long': -2.1327, 'officialname': 'Longton'}, {lat: 52.7661, 'long': -0.886, 'officialname': 'Melton Mowbray'}, {lat: 52.606, 'long': -1.9179, 'officialname': 'Aldridge'}, {lat: 53.5452, 'long': -2.3999, 'officialname': 'Farnworth'}, {lat: 51.552, 'long': -0.097, 'officialname': 'Highbury'}, {lat: 53.3761, 'long': -2.1897, 'officialname': 'Chaddle Hulme'}, {lat: 54.62, 'long': -1.58, 'officialname': 'Newton Aycliffe'}, {lat: 52.4299, 'long': -1.9355, 'officialname': 'Bournville'}, {lat: 52.009, 'long': -0.789, 'officialname': 'Shenley Brook End'}, {lat: 54.85, 'long': -1.83, 'officialname': 'Consett'}, {lat: 51.3211, 'long': -0.1386, 'officialname': 'Coulson'}, {lat: 52.566, 'long': -2.0728, 'officialname': 'Bilston'}, {lat: 52.7001, 'long': -2.5157, 'officialname': 'Wellington'}, {lat: 54.663, 'long': -1.676, 'officialname': 'Bishop Auckland'}, {lat: 52.395, 'long': -1.979, 'officialname': 'Longbridge'}, {lat: 52.614, 'long': -2.004, 'officialname': 'Bloxwich'}, {lat: 51.5557, 'long': 0.2512, 'officialname': 'Uxminster'}, {lat: 53.321, 'long': -3.48, 'officialname': 'Rhyll'}, {lat: 52.267, 'long': -2.153, 'officialname': 'Droitwich'}, {lat: 53.5355, 'long': -2.5658, 'officialname': 'Hindley'}, {lat: 53.549, 'long': -2.529, 'officialname': 'Westhoughton'}, {lat: 51.3589, 'long': 1.4394, 'officialname': 'Broadstairs'}

Utilisez une tranche dans la boucle for pour la tester :

Ajouter [10:20] après le nom de la liste signifie qu'on ne parcourra que les éléments aux positions 10 à 19

```
for rec in listofdicstocsv[10:20]:
```

```
    # Lire la latitude, la longitude et le nom du dictionnaire actuel
```

```
    lat = float(rec.get('lat', float('nan'))) # 'nan' utilisé comme substitut si la valeur est manquante
```

```
    lon = float(rec.get('long', float('nan')))
```

```
    officialname = rec.get('officialname', 'unknown')
```

```
    # Si la latitude ou la longitude est manquante, ignorer cet emplacement
```

```
    if math.isnan(lat) or math.isnan(lon):
```

```
        print(f"skipping '{officialname}': missing lat/long")
```

```
        continue
```

**Comment faire ça
pour des zones de
tailles différentes ?**

**Nous avons besoin
d'échelles différentes :
Grande conurbation,
zone plus petite**

Temps de 'Vibe coding'?

Utilisez le meta prompting pour demander à l'IA de concevoir le prompt d'abord

Incluez :

- Le contexte (journalisme)
- L'objectif
- **Instructions détaillées** (noms spécifiques, fichiers, etc.)

Vous êtes mon mentor, un journaliste de données avec plus d'une décennie d'expérience dans le domaine. Vous avez une **connaissance statistique avancée** ainsi qu'un **scepticisme sain** dans le traitement des données comme des sources humaines.

Je vous demanderai de l'aide pour du codage — vous êtes heureux de me guider, mais **vous ne voulez pas que je perde mes compétences et devienne trop dépendant** de vous, donc vos conseils seront toujours conçus pour **me forcer à réfléchir par moi-même, à acquérir de nouvelles compétences et concepts, et à les pratiquer.**

Lorsque vous fournissez du code, **ajoutez des commentaires expliquant chaque étape** comme si c'était à une personne sans expérience de codage.

préférer du code simple sur plusieurs lignes plutôt que du code complexe en une ou deux lignes.

Signalez toute hypothèse que vous formulez concernant les données ou ma question. Avertissez-moi si la question ne contient pas suffisamment d'informations pour y répondre avec précision, ou si la réponse pourrait être trompeuse sans contexte supplémentaire.

Ci-joint du code Python qui génère des images pour des cartes centrées sur des centaines d'emplacements. **Adaptez-le pour qu'il génère désormais ces images à deux échelles différentes.**

Commentez le code pour mettre en évidence la section qui fait cela, afin que je puisse l'ajuster et essayer différents niveaux de zoom.

Paramètres

[https://github.com/paulbradshaw/QGIS_param/blob/main/qgis_take_pix_t
woSize.py](https://github.com/paulbradshaw/QGIS_param/blob/main/qgis_take_pix_t
woSize.py)

La section définissant les paramètres **scale** d'échelle

DÉFINIR LES ÉCHELLES DE LA CARTE :

```
MAP_SCALES = [  
    175000,    # original scale  
    120000    # zoomed-in version  
]
```

La section utilisant ces paramètres

```
# NOUVEAU : BOUCLE À TRAVERS PLUSIEURS ÉCHELLES
```

```
for MAP_SCALE in MAP_SCALES:
```

```
    # Créez un rectangle temporaire autour du point pour le centrer.
```

```
    map_item.setExtent(
```

```
        rect_around(x, y, 1000, 1000)
```

```
)
```

```
    # Appliquer l'échelle actuelle
```

```
    map_item.setScale(MAP_SCALE)
```

```
    #ajouter une étiquette avec ces informations
```

```
    label.setText(f"{officialname} | scale 1:{MAP_SCALE:,}")
```

...et ajout au nom de fichier

```
# Utilisez cela pour déterminer le chemin d'enregistrement.  
out_path = (  
    f"{out_prefix}"  
    f"{safe_name}_{MAP_SCALE}.png" # NEW: ADD SCALE TO FILENAME  
)
```

L'ami critique :
Ce que l'IA devrait vous
dire

Points à considérer

- **Conversion entre le SCR de la carte de base et celui du fichier de formes**
- **Conversion entre la taille de mise en page (mm) et la taille d'image (px)**
- **Export pour impression/résolution 200 dpi**
- **Catégories d'échelle plutôt que chiffres bruts (et plus de deux)**

Essayez ceci :
Autres fichiers
facultatifs dans le dépôt

Points clés à retenir

- Si vous devez le faire plus d'une fois...
- Le **vibe coding** est amusant mais le *mentor coding* est plus enrichissant

Ressources supplémentaires :

- https://github.com/paulbradshaw/QGIS_param
- Tim Green: [Vibe Coding Threatens Journalism: Why Newsrooms Need Governance Now](#)
- [FT newsletter](#) sur leur méthodologie utilisant l'IA pour aider au codage dans la cartographie des zones à risque d'inondation